REAL TIME CHATTER APPLICATION

A high-performance, full-stack real-time messaging platform built for low latency and reliability.

FASTAPI | WEBSOCKETS | SQLMODEL / SQLITE

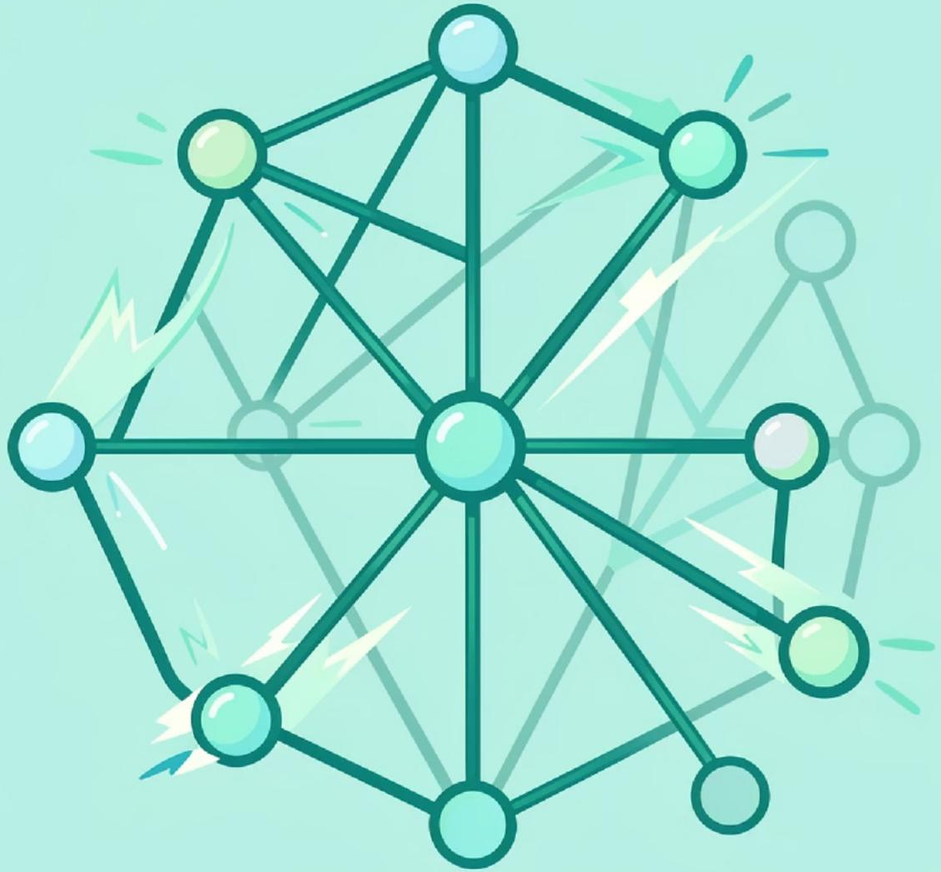
Introduction & Project Goals

The Vision

Seamless, low-latency communication with secure sessions and durable history for web and mobile clients.

Key Objectives

- Full-duplex instant messaging via WebSockets
- Persistent chat history stored reliably
- Secure authentication and session management
- Responsive UI across device form factors



System Architecture (Overview)

Design choices prioritise async IO, minimal operational overhead, and predictable performance under concurrent connections.

Backend

FastAPI (async) for request handling and WebSocket endpoints.

Real-Time Layer

Raw WebSocket connections provide persistent, bidirectional streams.

Database

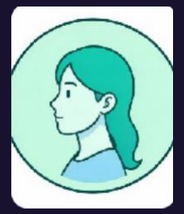
SQLModel + SQLite for relational clarity and simple deployment.

Frontend

Vanilla JS, HTML5, CSS3 — lightweight and highly optimised for rendering chat UI.

Database Schema Design

Schema designed to balance read performance for recent chats and safe writes for auditability.



RegisteredUsers

Stores persistent identity: email, username, hashed password, profile metadata.

ActiveUsers

Holds live session tokens (UUIDs), last seen timestamp and socket references.

Messages

Append-only log: message id, sender_id, content, channel, created_at for audit and syncing.

User Onboarding & Authentication

01

Registration

Passwords hashed with Bcrypt before persistence; email/username uniqueness enforced at model layer.

02

Login Flow

Credentials verified; server issues a UUID4 token tied to ActiveUsers for session validity checks.

03

Session Persistence

Token stored in browser localStorage, enabling page refresh resilience and re-authentication on reconnect.

The Real-Time Engine — Connection Manager

ConnectionManager centralises socket state and ensures rapid, consistent broadcasting while maintaining security checks.

1

State Management

In-memory dictionary of active socket objects keyed by user_id or token for O(1) lookup.

2

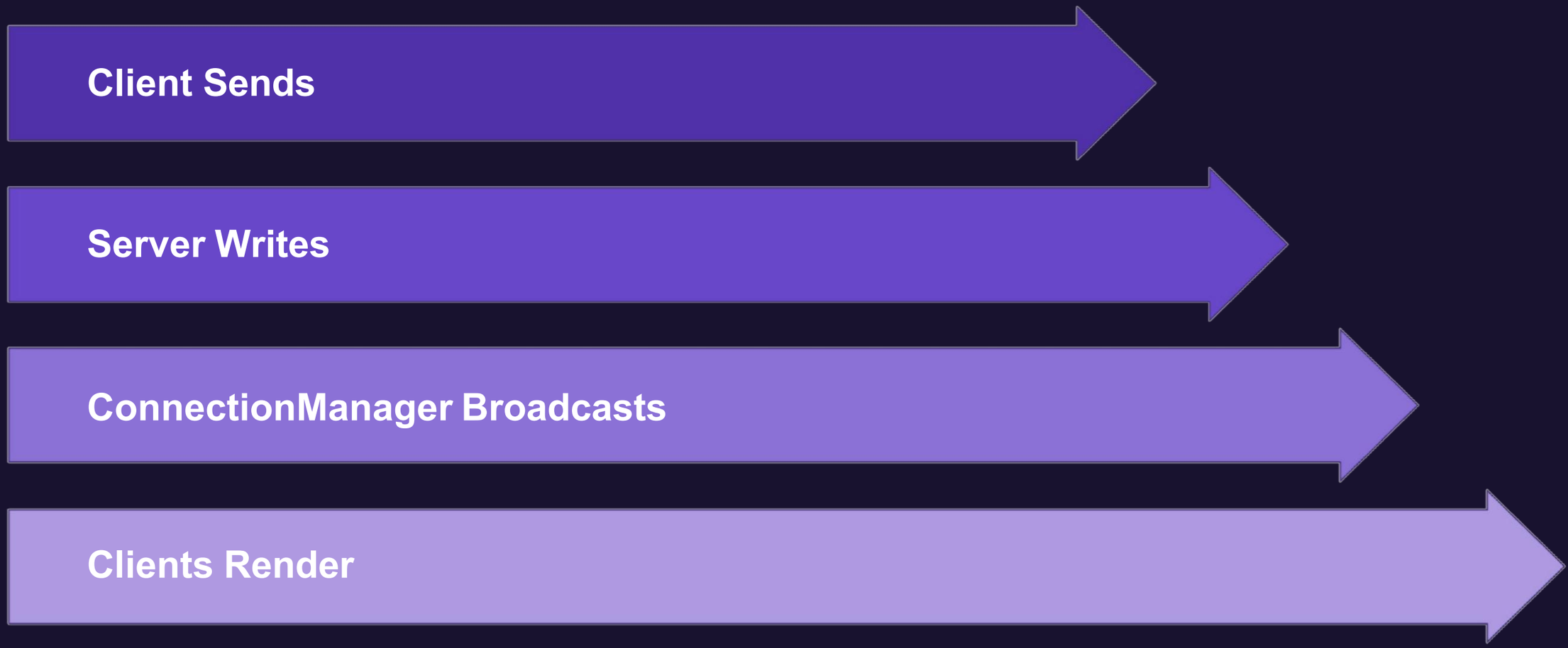
Handshake

On connect: validate token, confirm ActiveUsers entry, attach socket to ConnectionManager .

3

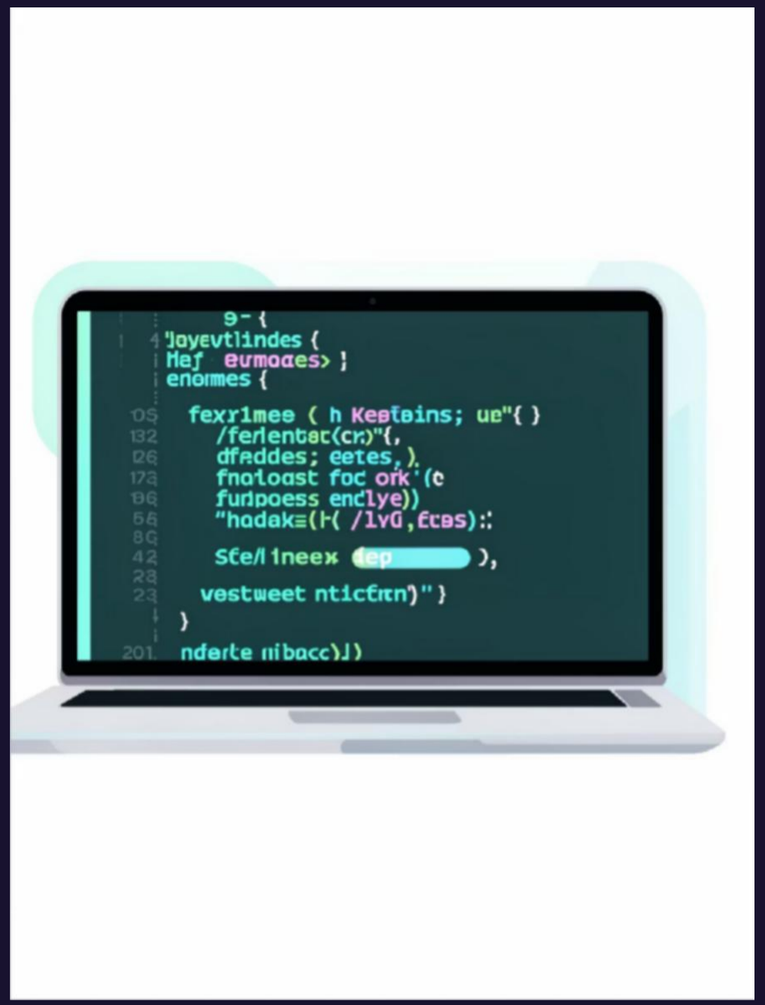
Broadcasting

Message pushed to relevant sockets; non-blocking sends with asyncio ensure millisecond delivery.



The lifecycle shows durability and real-time delivery: write → confirm → broadcast → render. This order preserves history and ensures eventual consistency across clients.

UI/UX Design & Animations



Responsive Layout

CSS Flexbox ensures the chat container adapts across breakpoints with touch-friendly targets.

Message Styling

Sent messages align right with a distinct accent (#26A688); received messages align left in neutral greys for clear differentiation.

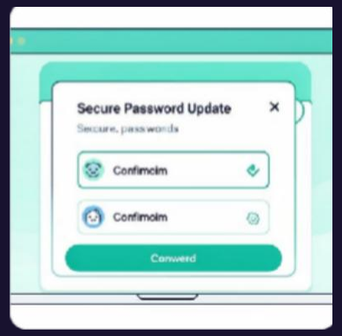
Smooth Transitions

Subtle @keyframes make incoming messages slide and fade—improving perceived performance.

Advanced Features

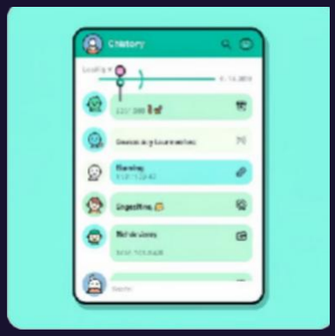
Profile Management

Users upload avatars; images stored server-side with secure paths; localStorage caches small metadata for UI speed.



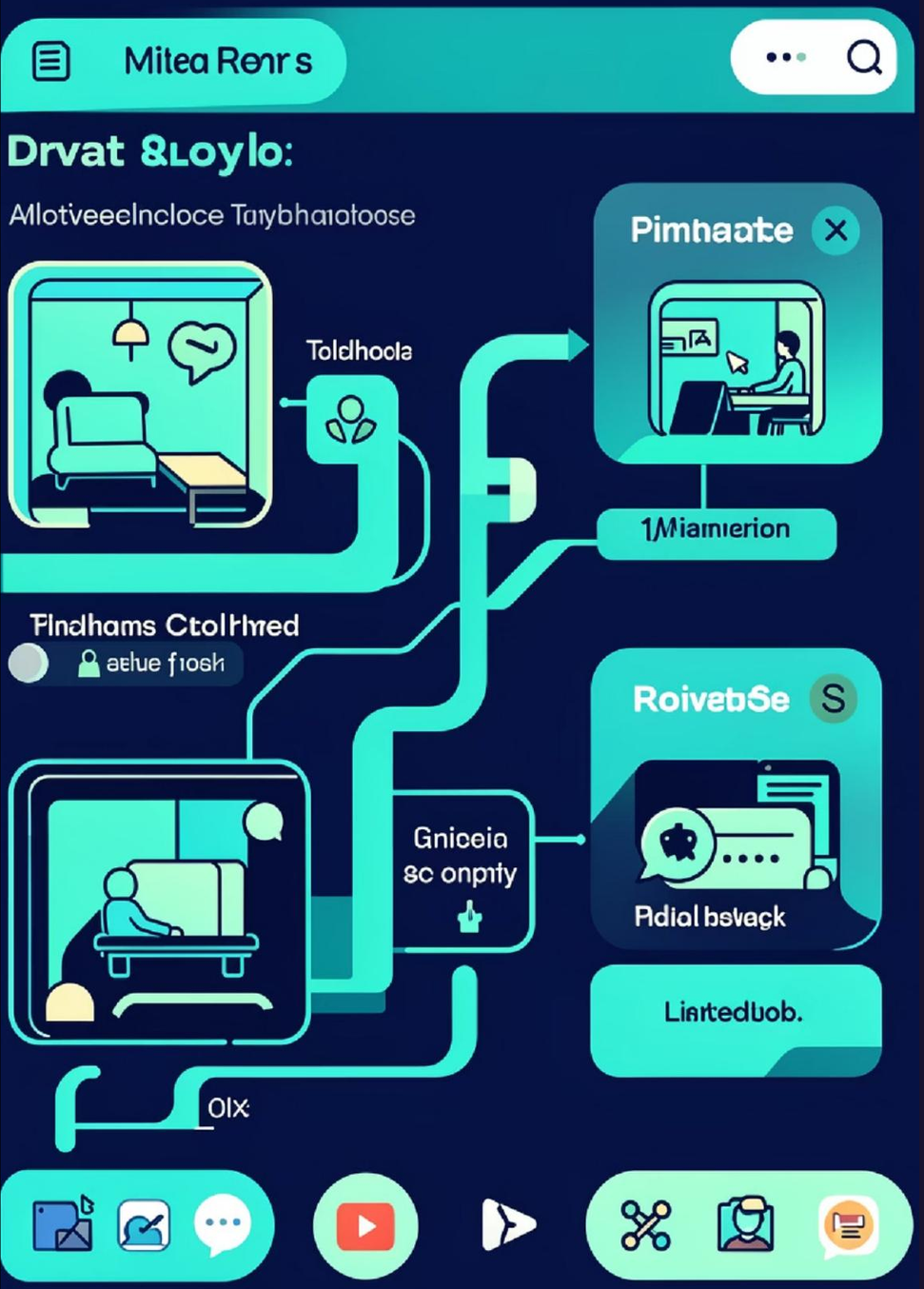
Password Security

Interactive modal for password updates; reauth required and Bcrypt rehash performed on change.



Chat History

On login the server returns the most recent 100+ messages; incremental pagination supports infinite scroll.



Conclusion & Future Scope

Summary

A production-ready architecture delivering secure, low-latency messaging with persistent storage and lightweight clients.

Planned Enhancements

- Private 1-on-1 rooms and ACLs
- Real-time typing & presence indicators
- Media sharing (images, video, voice) with streaming and CDN support

Next Steps

Prototype testing under concurrent load, implement end-to-end encryption options, and productionise with a durable RDBMS and horizontal scaling plan.

Q&A — Invite technical questions on implementation, performance assumptions and scaling trade-offs.